

Iterative Improvement Algorithms for the Linear Ordering Problem

INFO-H-413 — Heuristic Optimization — Implementation Exercise 1

Université Libre de Bruxelles — 2026

Luis Brunard (577721)

1. Introduction

The Linear Ordering Problem (LOP) asks the following question: given an $n \times n$ matrix C of weights, find an ordering (permutation) of the n elements that maximises the sum of the weights that appear **above** the diagonal once the matrix rows and columns are reordered accordingly. Formally, we look for a permutation π that maximises:

$$f(\pi) = \sum_{i < j} c_{\pi(i), \pi(j)}$$

Intuitively, we want to place element i before element j whenever $c_{ij} > c_{ji}$, i.e., when “choosing i first” is more valuable. The problem is NP-hard, which means there is no known algorithm that solves it exactly in polynomial time for large instances. Heuristics are therefore needed.

This exercise focuses on **local search**: we start from some initial solution and keep making small changes that improve it, until no further improvement is possible. This final state is called a **local optimum**. We study how the choice of starting solution, the type of change allowed (neighbourhood), and the strategy for choosing which change to apply (pivoting rule) affect the quality of the result and the time needed to find it.

2. Problem Description

2.1. Neighbourhood Structures

A **neighbourhood** defines the set of solutions we can reach in one step from the current solution. Three neighbourhoods are considered.

Transpose. Swap two **adjacent** elements. From a permutation of length n , we can make $n - 1$ such swaps. The gain Δ of swapping positions i and $i + 1$ is simply:

$$\Delta_T(i) = c_{\pi(i+1), \pi(i)} - c_{\pi(i), \pi(i+1)}$$

This is $O(1)$

Exchange. Swap any two elements, not necessarily adjacent. There are $\binom{n}{2}$ possible pairs. Computing the gain requires scanning all elements between the two swapped positions, so it costs $O(n)$.

Insert. Remove an element from position i and reinsert it at position j , shifting everything in between. This gives $n(n - 1)$ possible moves (both forward and backward insertions). Each gain evaluation costs $O(n)$.

2.2. Pivoting Rules

Once the neighbourhood is fixed, we need a strategy to choose which improving move to apply.

- **First-improvement (FI)**: apply the first move with a positive gain and restart the scan immediately.
- **Best-improvement (BI)**: scan the **entire** neighbourhood, then apply the move with the highest gain.

2.3. Initialisation

- **Random**: generate a random permutation.
- **Chenery–Watanabe (CW)**: rank each element by its row-sum score $R_i = \sum_{j \neq i} c_{ij}$ and sort in decreasing order. This greedy heuristic puts elements that “benefit most from being first” at the front of the permutation.

3. Algorithms

3.1. Iterative Improvement (Exercise 1.1)

The basic algorithm is:

- Generate an initial solution π_0 .
- While an improving neighbour exists:
 - (First-improvement) Apply the first move with $\Delta > 0$; restart scan.
 - (Best-improvement) Apply the move with the largest $\Delta > 0$.
- Return the local optimum π^* .

Combining 2 pivoting rules with 3 neighbourhoods gives 6 algorithms, each run from 2 initial solutions = **12 configurations**.

3.2. Variable Neighbourhood Descent (Exercise 1.2)

The idea behind VND is simple: a local optimum for one neighbourhood might not be optimal for another. So instead of giving up when we get stuck, we switch to a different neighbourhood. If the new neighbourhood finds an improvement, we restart from the first neighbourhood. We stop only when **none** of the neighbourhoods can improve the solution.

We use first-improvement only, starting from the CW solution. Two orderings of the three neighbourhoods are tested:

- **VND-TEI**: Transpose $\rightarrow$ Exchange $\rightarrow$ Insert
- **VND-TIE**: Transpose $\rightarrow$ Insert $\rightarrow$ Exchange

4. Statistical Testing Method

4.1. Standard deviation and error bars

The **standard deviation** (SD) measures how spread out a set of values is around their average. A small SD means most results are close to the average; a large SD means they vary a lot across instances. In the graphs, the small horizontal segments on each bar are **error bars** showing ± 1 SD. A short bar means the algorithm is consistent; a long bar means results depend heavily on the instance. For example, Insert-First (CW) has mean RPD = 1.59% and SD = 0.33%, so most instances produced results between 1.26% and 1.92% [1].

4.2. Wilcoxon test and p-value

Comparing average RPD values is not sufficient on its own — the difference could be due to a few unusual instances rather than a genuine trend. We therefore use the **Wilcoxon signed-rank test** [2], a standard non-parametric test for paired data: for each instance, we compute the RPD difference between the two algorithms, rank these differences by absolute value, and check whether one algorithm wins consistently. The result is a **p-value**: the probability of seeing a difference this large if the two algorithms were actually equal. We use the threshold $\alpha = 0.05$ — if $p < 0.05$, the difference is declared **statistically significant** [3]. The test was run in R using the built-in `wilcox.test` function [4].

5. Results and Analysis

All 78 benchmark instances (sizes 150 and 250) were solved by each of the 14 algorithm configurations (12 for Ex 1.1 + 2 VND). Each instance was solved once. Solution quality is measured with the **Relative Percentage Deviation** (RPD) from the best-known solution:

$$\text{RPD} = \frac{\text{BestKnown} - \text{FinalCost}}{\text{BestKnown}} \times 100$$

An RPD of 0 means we matched the best-known value. A positive RPD means our solution is that many percent below the best known (so **lower RPD is better**).

5.1. Exercise 1.1 — Iterative Improvement Results

Figure 1 gives an immediate visual overview of solution quality for the Insert, Exchange and VND configurations (Transpose is excluded from this chart as its RPD of 19–35% would compress the scale). Table 1 gives the full numerical results for all 12 configurations.

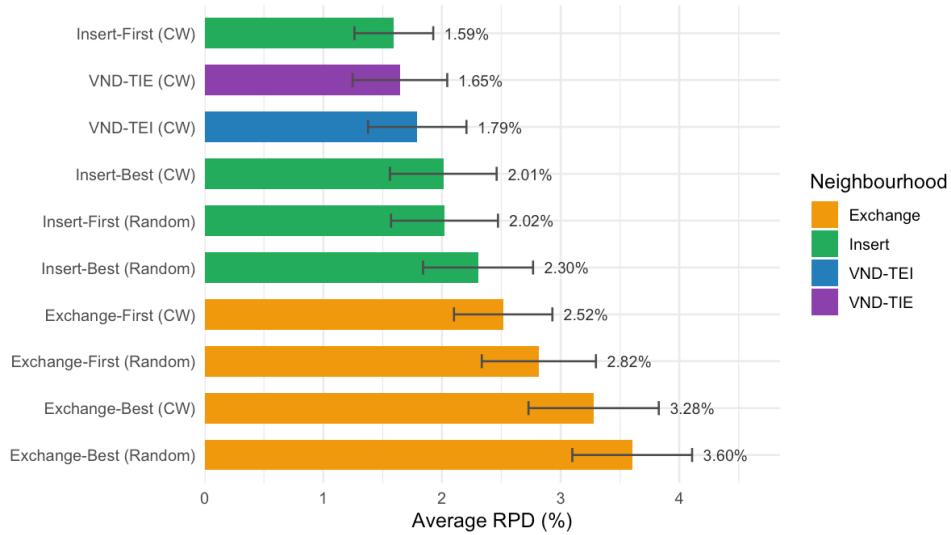


Figure 1: Average RPD (%) for Insert, Exchange and VND configurations, sorted from best (top) to worst (bottom). Each bar shows the average over 78 instances; the horizontal black segments are **error bars** representing ± 1 standard deviation — they show how much the RPD varies across instances. A short bar means the algorithm behaves consistently; a long bar means performance depends heavily on the instance. The values printed to the right of each error bar are the exact average RPD. Transpose is excluded (RPD 19–35%) to keep the scale readable; it appears in Table 1. Colours indicate the neighbourhood: green = Insert, orange = Exchange, blue = VND-TEI, purple = VND-TIE.

Algorithm	Init	Avg RPD	SD	Avg Time (s)	Total Time (s)
Insert — First	CW	1.59	0.33	2.99	233.4
Insert — Best	CW	2.01	0.45	0.79	61.7
Insert — First	Ran- dom	2.02	0.45	4.28	333.8
Insert — Best	Ran- dom	2.30	0.46	0.85	66.3
Exchange — First	CW	2.52	0.41	3.97	309.7
Exchange — First	Ran- dom	2.82	0.48	4.87	380.1
Exchange — Best	CW	3.28	0.55	0.70	54.3
Exchange — Best	Ran- dom	3.60	0.50	0.76	59.3
Transpose — Best	CW	19.25	1.89	0.006	0.45
Transpose — First	CW	19.38	1.92	0.003	0.25
Transpose — Best	Ran- dom	34.47	3.80	0.007	0.55
Transpose — First	Ran- dom	34.61	3.80	0.004	0.33

Table 1: Average RPD (%), standard deviation, and computation times for all 12 configurations over 78 instances. Green = best group, yellow = intermediate, red = transpose (poor quality).

5.1.1. Observations

Neighbourhood choice is the dominant factor. The insert neighbourhood is clearly best: it achieves RPD around 1.6–2.3%, versus 2.5–3.6% for exchange and an unacceptably high 19–35% for transpose. Transpose simply cannot escape poor solutions because it can only make tiny adjacent swaps; it gets trapped in deep local optima.

CW initialisation is significantly better than random. For every neighbourhood and pivoting rule, the CW starting solution leads to better final results. The difference is most striking for transpose (19% vs 34% RPD), confirming that the CW heuristic provides a starting point already close to a good solution.

First-improvement beats best-improvement in quality — but not in speed. For insert, first-improvement (RPD 1.59% from CW) reaches a better solution than best-improvement (RPD 2.01% from CW). This is a somewhat surprising result. The reason is likely that first-improvement makes finer, more cautious moves, allowing it to explore a richer part of the search space before converging, whereas best-improvement makes large jumps that can overshoot.

Best-improvement is much faster despite examining more neighbours per step. Insert-BI from CW takes only 0.79 s on average, while Insert-FI from CW takes 2.99 s. Best-improvement converges in far fewer iterations because each step makes a bigger improvement, which more than compensates for the extra scanning work.

5.1.2. Statistical Tests

Pairwise Wilcoxon signed-rank tests were applied at significance level $\alpha = 0.05$. A representative selection of results is shown in Table 2.

Algorithm A	Algorithm B	p-value	Significant?
Insert-First (CW)	Transpose-First (CW)	< 0.001	Yes
Insert-First (CW)	Exchange-First (CW)	< 0.001	Yes
Insert-First (CW)	Insert-Best (CW)	< 0.001	Yes
Insert-First (CW)	Insert-First (Random)	< 0.001	Yes
Insert-Best (CW)	Insert-First (Random)	0.9404	No
Exchange-First (CW)	Exchange-Best (CW)	< 0.001	Yes
Transpose-First (CW)	Transpose-First (Random)	< 0.001	Yes

Table 2: Selected pairwise Wilcoxon signed-rank test results ($\alpha = 0.05$). All 66 pairs were tested; nearly all are significant except the highlighted case.

Almost all differences are statistically significant ($p < 0.001$). The one interesting exception is **Insert-Best (CW) vs Insert-First (Random)** ($p = 0.94$): despite very different strategies, these two configurations happen to produce solutions of indistinguishable quality (RPD 2.01% vs 2.02%). This shows that a good initialisation (CW) can compensate for a weaker pivoting rule (best-improvement), and vice versa.

Figure 2 confirms visually that the CW initialisation consistently produces better and tighter RPD distributions than the random start, for both Insert and Exchange neighbourhoods.

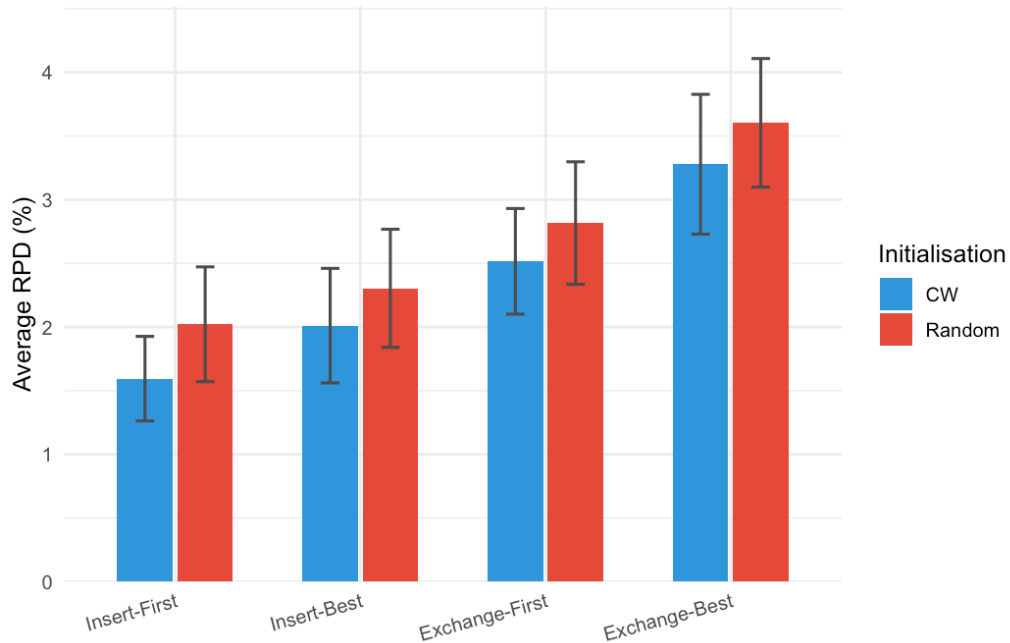


Figure 2: Average RPD (%) for CW initialisation (blue) and random initialisation (red), for each neighbourhood and pivoting rule combination. Error bars show ± 1 SD. For every configuration, the blue bar is lower than the red bar, confirming that the CW starting solution consistently leads to better final solutions.

5.2. Exercise 1.2 – VND Results

Table 3 and Figure 3 show the performance of both VND orderings, starting from the CW solution.

Algorithm	Init	Avg RPD	SD	Avg Time (s)	Total Time (s)
VND-TIE	CW	1.65	0.40	3.24	252.4
VND-TEI	CW	1.79	0.42	4.86	379.1

Table 3: VND results starting from the CW solution.

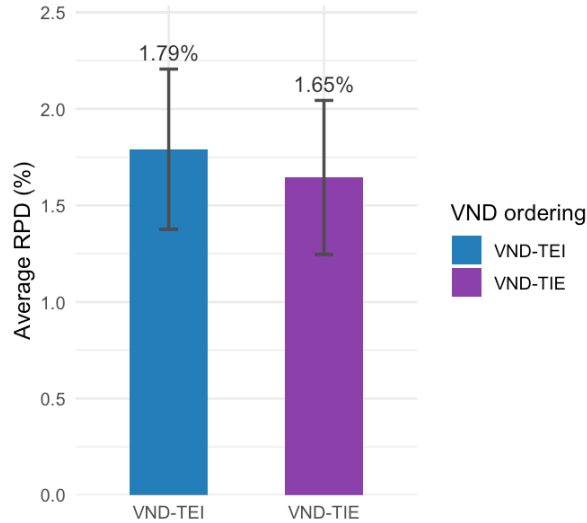


Figure 3: Average RPD (%) for VND-TEI and VND-TIE. VND-TIE achieves a lower and more consistent RPD. The difference is statistically significant (Wilcoxon test, $p = 0.0074$).

Both VND variants give better results than any single-neighbourhood algorithm. For reference, the best individual algorithm was Insert-First (CW) with RPD 1.59%, which is very close to VND-TIE at 1.64%. This small gap suggests that the insert neighbourhood alone already captures most of the improvement available.

VND-TIE is better than VND-TEI both in quality (1.64% vs 1.79% RPD) and in speed (3.24 s vs 4.86 s per instance), and this difference is confirmed significant by the Wilcoxon test ($p = 0.0074$). A likely explanation is that placing insert **before** exchange (as in TIE) allows the most powerful neighbourhood to act early, whereas in TEI, exchange runs first and may lead to a local optimum that insert then struggles to improve upon.

5.3. Execution Time and Complexity

5.3.1. Why do some algorithms take much longer than others?

The execution times in Table 1 vary by several orders of magnitude — from under 1 millisecond (transpose) to nearly 5 seconds (exchange/insert first-improvement). This is directly explained by the theoretical complexity.

Each **iteration** of the local search requires scanning the neighbourhood to find an improving move. The cost of one iteration depends on two factors: the size of the neighbourhood (how many moves to check) and the cost of evaluating each move.

Neighbourhood	Size	Cost per Δ	Cost per iteration
Transpose	$n - 1$	$O(1)$	$O(n)$
Exchange	$O\left(\frac{n^2}{2}\right)$	$O(n)$	$O(n^3)$ for BI, $O(n^2)$ avg for FI
Insert	$O(n^2)$	$O(n)$	$O(n^3)$ for BI, $O(n^2)$ avg for FI

Table 4: Theoretical cost per iteration of local search for each neighbourhood.

- **Transpose** is $O(n)$ per iteration because each gain is $O(1)$ and there are only $n - 1$ adjacent pairs to check. This explains why it runs in under 10 ms even for $n = 250$. However, its small neighbourhood also explains its very poor solution quality: it can only make tiny moves.
- **Exchange and Insert** are $O(n^3)$ per iteration for best-improvement (scan $O(n^2)$ pairs, each costing $O(n)$). For first-improvement, the average cost is lower because the scan stops as soon as an improvement is found – but in the worst case (near a local optimum) almost the full neighbourhood must be scanned before stopping.

Figure 4 shows this trade-off visually: the bottom-left corner is the ideal (fast and accurate). Insert-FI (CW) is the clear winner on quality; Insert-BI and Exchange-BI are faster but sacrifice quality.



Figure 4: Quality–time for all configurations (Insert, Exchange and VND).

5.3.2. Why is best-improvement faster than first-improvement for insert?

This counterintuitive result (BI: 0.79 s vs FI: 2.99 s for Insert-CW) is explained by the number of **iterations**. Best-improvement makes the largest possible improvement at each step, so it converges in far fewer iterations. Although each iteration is slower (full scan), the total number of iterations is much smaller, and the product (iterations $\times$ cost per iteration) is lower for BI. First-improvement makes smaller moves and therefore requires many more iterations to reach a local optimum.

5.3.3. Effect of instance size

Going from size $n = 150$ to $n = 250$, the per-iteration cost grows roughly as $\left(\frac{250}{150}\right)^3 \approx 4.6$ for exchange and insert (BI). This cubic scaling is confirmed by the data: exchange-BI (CW) takes about 0.70 s for the average instance, which includes both sizes mixed.

5.3.4. Effect of initialisation on time

Starting from a CW solution saves time because the algorithm is already close to a local optimum and needs fewer iterations to converge. For insert-FI, the CW start (2.99 s) is notably faster than the random start (4.28 s), a speedup of about 1.4×.

5.3.5. Possible optimisations

- **Don't-look bits:** skip elements that have not been involved in a recent improving move, reducing the effective neighbourhood size.
- **Incremental delta table:** after applying a move, only update the affected gain values instead of recomputing all of them.
- **Candidate lists:** precompute a short list of the most promising move pairs for each element and only scan those.

6. Conclusion

This exercise showed that local search for the LOP is sensitive to all three design choices studied. The neighbourhood structure has the biggest impact: insert clearly dominates exchange, and both completely dominate transpose. The CW initialisation gives significantly better results than random in every configuration. The choice of pivoting rule has a smaller but still significant effect: for the insert neighbourhood, first-improvement produces marginally better quality at the cost of longer running time.

VND provides a modest improvement over the best single-neighbourhood algorithm (Insert-First CW), and VND-TIE is the best overall configuration, combining good solution quality (RPD 1.64%) with reasonable computation time. The order in which neighbourhoods are combined matters: placing insert second rather than third (TIE vs TEI) is both faster and more effective.

The algorithms built in this exercise will serve as building blocks for the second implementation exercise, where metaheuristic techniques will be applied to further improve solution quality.

References

- [1] Wikipedia contributors, “Standard deviation.” [Online]. Available: https://en.wikipedia.org/wiki/Standard_deviation
- [2] Wikipedia contributors, “Wilcoxon signed-rank test.” [Online]. Available: https://en.wikipedia.org/wiki/Wilcoxon_signed-rank_test
- [3] Wikipedia contributors, “P-value.” [Online]. Available: <https://en.wikipedia.org/wiki/P-value>
- [4] R Core Team, “wilcox.test: Wilcoxon Rank Sum and Signed Rank Tests.” [Online]. Available: <https://stat.ethz.ch/R-manual/R-devel/library/stats/html/wilcox.test.html>